

SYSTEM ANALYST

presented by Ayub

Class Diagram

Мы знаем ООП — теперь научимся его рисовать

В прошлом уроке мы разобрали концепции:

ООП-концепция	Что это
Класс	Шаблон объекта
Атрибуты	Данные класса
Методы	Поведение класса
Наследование	Передача свойств

Class Diagram — стандартный язык (нотация) для изображения этих концепций.

Как музыкальные ноты: все музыканты мира читают одинаково. Разработчики читают Class Diagram так же — в любой стране.

Ты — аналитик. Объясняешь разработчику структуру системы.

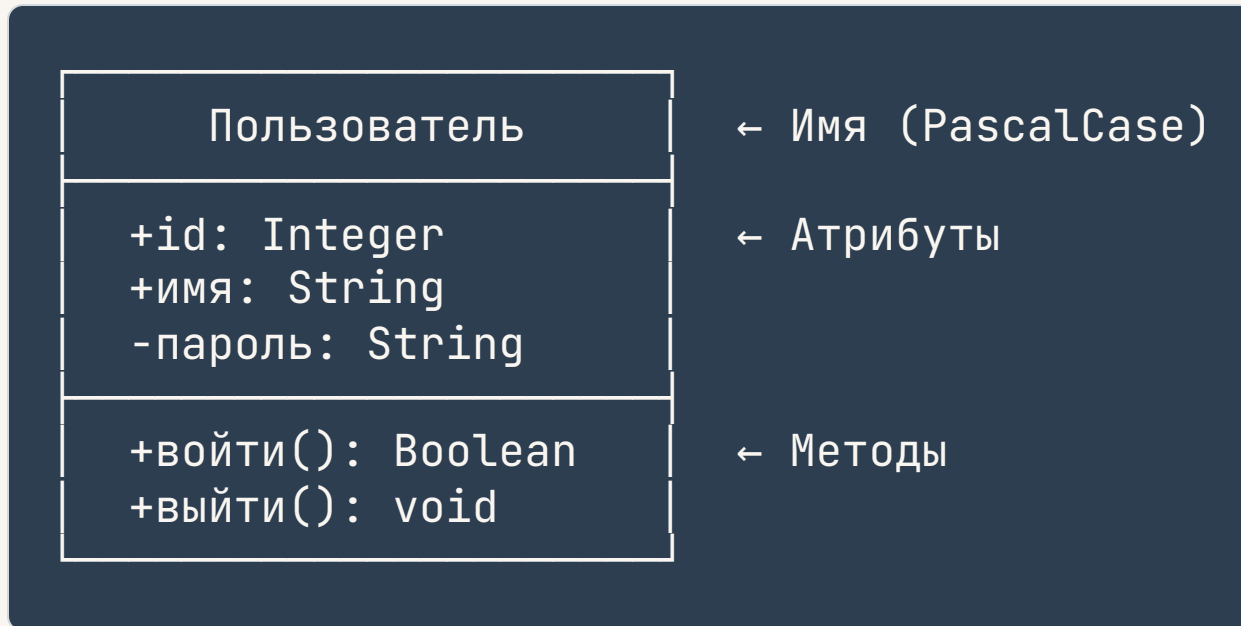
Говоришь: "Есть класс Пользователь, у него есть имя и пароль, он умеет войти в систему."

Разработчик записывает по-своему. Аналитик в другой команде записывает иначе.

Как нарисовать класс так, чтобы любой разработчик в любой стране понял структуру без объяснений?

Структура класса: три секции

Прямоугольник, разделённый на три части:



- Секция 1: **ИМЯ** — всегда PascalCase
(Пользователь, OrderItem)

Секция 1:
Секция 2:
Секция 3:

Visibility: кто видит атрибут или метод

Символ	Название	Кто имеет доступ
+	public	Все снаружи
-	private	Только сам класс
#	protected	Класс + наследники

Для SA на практике:

- + — входит в API-документацию, виден клиенту
- - — внутренняя реализация, скрыта от других
- # — виден дочерним классам, но не всем

-пароль: *String* — пароль не возвращается через API. Только сам класс умеет с ним работать.

Типы данных в атрибутах

Тип	Что хранит	Пример
String	Текст	+имя: String
Integer	Целое число	+количество: Integer
Decimal	Число с дробью	+цена: Decimal
Boolean	Да / Нет	+активен: Boolean
Date	Дата	+датаРегистрации: Date

Зачем SA это знать?

- Тип данных → тип поля в базе данных
- String → VARCHAR, Decimal → DECIMAL(10,2), Boolean → BOOLEAN

UML. Class Diagram

Class Diagram — мост между требованиями и реализацией. Тип данных задаёт разработчику конкретику

**Мы знаем из ООП: есть Наследование, Ассоциация,
Агрегация, Композиция.**

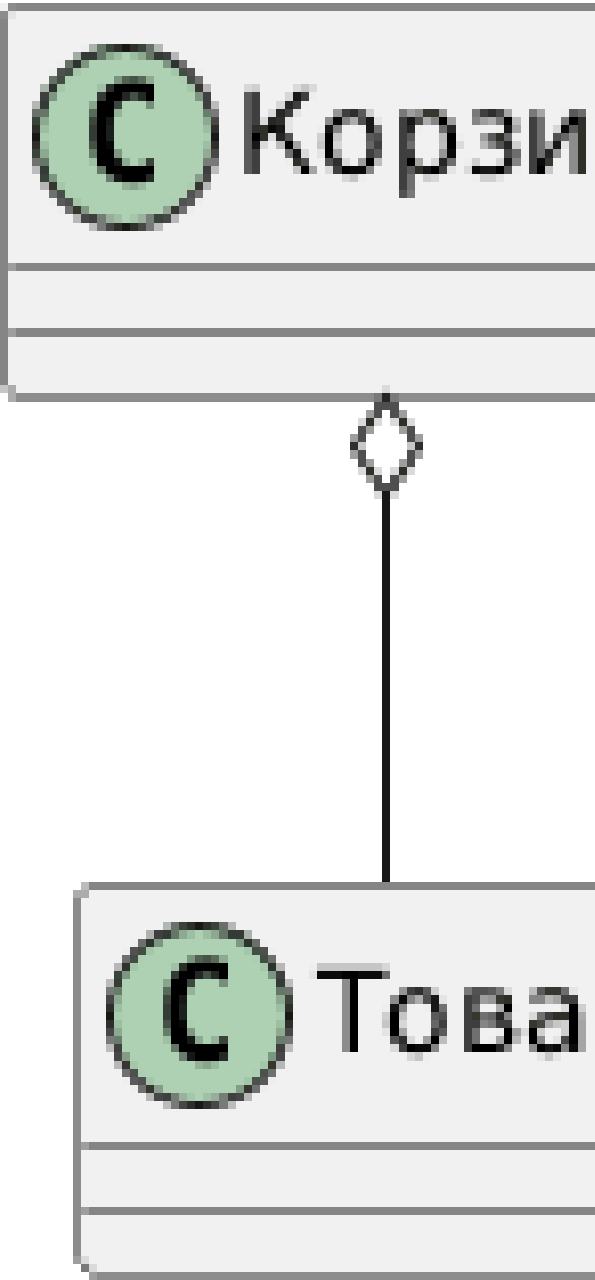
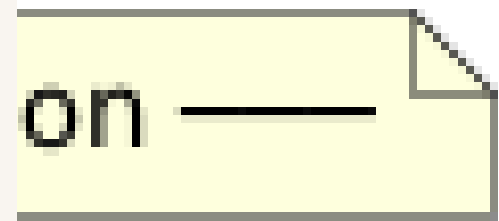
На словах легко. Но как их отличить на диаграмме?

**У каждого типа связи — свой символ. Как дорожные
знаки: увидел форму — знаешь смысл.**

4 типа связей: символы

Связь	Символ	Читается как
Inheritance	→ (полая стрелка)	"является подтипом"
Association	— (простая линия)	"связан с"
Aggregation	◇— (полый ромб)	"содержит, но живёт отдельно"
Composition	◆— (закрашенный ромб)	"содержит и владеет"

UML: Class Diagram



Multiplicity: сколько объектов участвует

Записывается у **ОБОИХ** концов линии:

Запись	Читается
1	Ровно один
0..1	Ноль или один (необязательный)
0..*	Ноль или много
1..*	Один или много (минимум один)

- * — сокращение для 0..*
- Числа пишутся рядом с классом, к которому **ОНИ ОТНОСЯТСЯ**



Читаем Multiplicity вслух

Разберём пошагово:

Покупатель "1" — "0..*" Заказ

1. Берём левый класс: **Покупатель**
2. Смотрим на правую кратность (0..*): **ноль или много Заказов**
3. Берём правый класс: **Заказ**
4. Смотрим на левую кратность (1): **ровно один Покупатель**

Итого: "1 Покупатель оформляет 0..* Заказов"

Заказ "1" — "1..*" Товар

→ "В 1 Заказе есть минимум 1 Товар"

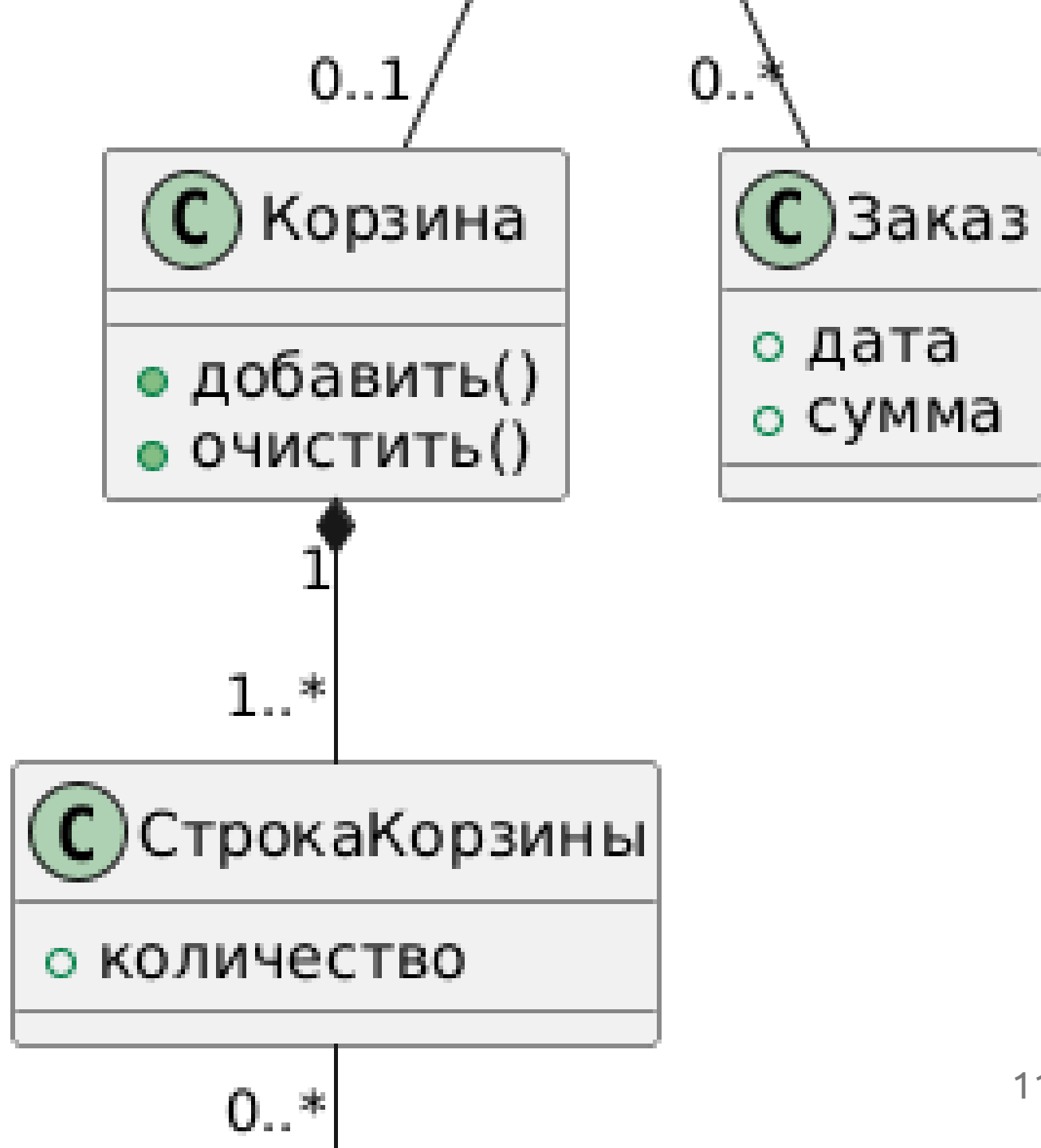
Module Check 1

1. Нарисуй (мысленно или на листе) класс **Товар** с 3 атрибутами и 1 методом
2. Что значит запись `-цена: Decimal`?
3. Какой символ обозначает **Composition**?
4. Как записать связь "один-ко-многим" между Курсом и Студентами?
5. Чем отличается `0..*` от `1..*`?

Полная диаграмма интернет-магазина

Все концепции вместе: Inheritance, Association, Aggregation, Composition + Multiplicity

- **Администратор** наследует **Пользователь** (Inheritance $\rightarrow$)
- **Пользователь** агрегирует **Корзину** — корзина необязательна (1 to 0..1)
- **Корзина** компонуется **СтрокиКорзины** — строки исчезают вместе с корзиной



Читаем диаграмму магазина вслух

Пройдёмся по каждой связи:

- 1 Пользователь имеет 0..1 Корзину — агрегация, корзина необязательна
- 1 Корзина содержит 1..* СтрокиКорзины — композиция, минимум одна строка
- 0..* СтрокиКорзины ссылаются на 1 Товар — один товар в разных строках
- 1 Пользователь оформляет 0..* Заказов — заказов может не быть
- 0..* Товаров относятся к 1 Категории — каждый товар в одной категории

Каждая связь отвечает на вопрос: "сколько?" и "как крепко привязаны?"

Inheritance в деталях

Администратор наследует Пользователь:

Администратор —> Пользователь

Стрелка направлена вверх — к родителю.

- Администратор **автоматически получает:** `+id`, `+имя`, `+email`, `+войти()`, `+выйти()`
- Администратор **добавляет своё:** `+уровеньДоступа`, `+удалитьПользователя()`
- В диаграмме у Администратора пишем **только новые атрибуты** — унаследованные не дублируем

SA использует Inheritance, когда классы разделяют общие данные, но имеют разное поведение. Администратор "является" Пользователем — но с расширенными правами.

Микропрактика: 8 минут

Нарисуй Class Diagram для онлайн-курсов (как Coursera)

Минимум 4 класса: **Студент, Курс, Урок, Сертификат**

Требования:

- Visibility символы (`+` / `-` / `#`) у каждого атрибута
- Тип данных у каждого атрибута (`String` , `Integer` , `Date` ...)
- Минимум 1 метод в каждом классе (со скобками)
- Связи между классами с кратностью с обеих сторон

Peer Review: проверяем диаграмму соседа

Чек-лист (3 минуты):

- У каждого класса **3 секции**: имя / атрибуты / методы?
- Имя класса в **PascalCase**?
- У каждого атрибута есть **Visibility** символ?
- У каждого атрибута указан **тип данных**?
- Методы записаны со **скобками** `()`?
- Кратность **с обеих сторон** каждой связи?
- Символы связей правильные:  vs  vs  ?

Верни лист с одной конкретной правкой и одной похвалой.

У тебя есть класс Товар с атрибутами: id, название, цена, описание, категорияId.

Эти данные нужно хранить в базе данных PostgreSQL.

Как бы выглядела таблица для хранения этих товаров?

Class → SQL: маппинг

Class Diagram	SQL
Класс	CREATE TABLE
Атрибут	Колонка
String	VARCHAR(255)
Integer	INT
Decimal	DECIMAL(10,2)
Boolean	BOOLEAN
Date	DATE / TIMESTAMP
<<PK>>	PRIMARY KEY
<<FK>>	REFERENCES

Association → Foreign Key

"Товар принадлежит Категории" → в

SQL: `categoria_id INT REFERENCES categories(id)`

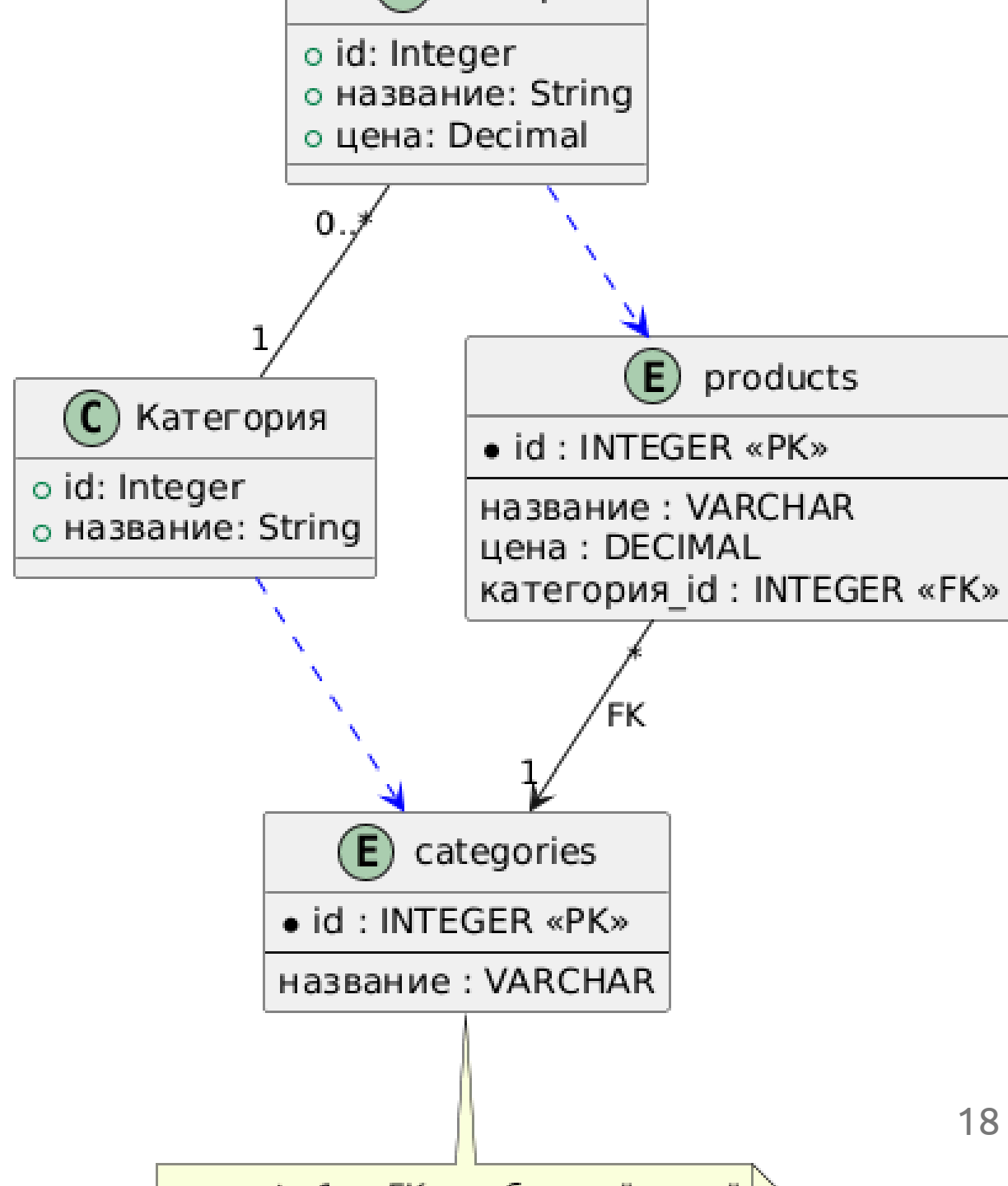
Правило: кратность many-to-1 → Foreign Key находится в таблице "many"

Товар "0..*" — "1" Категория

→ В таблице `products` появляется колонка `категория_id`

Many-to-many → промежуточная

таблица.



Class Diagram → ERD: шпаргалка

Class Diagram	ERD (Entity-Relationship)
Класс	Сущность (Entity)
Атрибут	Атрибут
<<PK>>	Первичный ключ
Multiplicity	Кардинальность (1:N, M:N)
Inheritance	Паттерн подтипов
Composition	Cascade delete

Главное отличие: ERD — про данные и хранение. Class Diagram — про поведение тоже (методы).

Типичные ошибки 1 и 2

Ошибка 1: методы в секции атрибутов

✗ -пароль: String
-войти(): Boolean ← метод в секции атрибутов!

✓ Атрибуты: -пароль: String
Методы: +войти(): Boolean

Ошибка 2: кратность не указана

✗ Покупатель — Заказ (сколько заказов? непонятно)
✓ Покупатель "1" — "0..*" Заказ

Нет кратности → разработчик будет гадать. Это баг в документации.

Типичные ошибки 3 и 4

Ошибка 3: перепутали и

- | | | |
|---|--|--|
| ✗ | Заказ  — СтрокаЗаказа | (Aggregation: строки живут без заказа?) |
| ✓ | Заказ  — СтрокаЗаказа | (Composition: строки принадлежат заказу) |

Тест: "Если удалить владельца — что будет с содержимым?"

- Исчезает → **Composition** 
- Остаётся → **Aggregation** 

Ошибка 4: слишком много деталей методов

- | | |
|---|--|
| ✗ | +оформитьЗаказ(userId: Integer, items: List): OrderDTO |
| ✓ | +оформитьЗаказ(): Order |

Module Check 2

1. Что такое **Multiplicity** и где она записывается?
2. Какой символ обозначает **Inheritance**?
3. Как класс с атрибутом `+id: Integer <<PK>>` превращается в SQL?
4. В чём разница между `+` и `-` visibility?
5. Как показать связь **many-to-many** между Студентом и Курсом?

Когда SA рисует Class Diagram

Ситуация	Инструмент
Описать структуру данных системы	Class Diagram
Описать роли и действия пользователей	Use Case Diagram
Описать бизнес-процесс (шаги, ответственные)	BPMN
Показать взаимодействие во времени	Sequence Diagram
Показать структуру базы данных	ERD

Class Diagram — лучший выбор, когда нужно договориться о структуре объектов до начала разработки.

Инструменты для Class Diagram

Инструмент	Тип	Особенность
draw.io	Визуальный	Бесплатный, в браузере, drag-and-drop
PlantUML	Текстовый	Диаграмма из кода — версионизируется в Git
Lucidchart	Визуальный	Платный, коллаборация в реальном времени
Enterprise Architect	Профессиональный	Полный UML, дорогой
dbdiagram.io	Специализированный	Только ERD, не Class Diagram

Рекомендация SA:

UML: Class Diagram

- Для команды → **draw.io** (бесплатно, все умеют)

Чек-лист хорошей Class Diagram

Проверь свою диаграмму:

- У каждого класса **3 секции**: имя / атрибуты / методы
- Имена классов в **PascalCase**, атрибуты в **camelCase**
- Каждый атрибут имеет **Visibility** символ (**+** / **-** / **#**)
- Каждый атрибут имеет **тип данных**
- Методы записаны со **скобками** **()**
- Кратность **у обоих концов** каждой связи
- Символы связей правильные: **◆** vs **◇** vs **→**
- Нет лишних параметров методов (SA пишет только имя и тип возврата)

Module Check финальный

1. Чем **Class Diagram** отличается от **Use Case Diagram**?
2. Мысленно нарисуй основные классы **Telegram**: какие классы? какие связи?
3. Какую связь ставишь между **Заказ** и **Товар** в интернет-магазине?
4. Что такое `<<PK>>` и `<<FK>>` ?
5. Почему `-пароль: String` правильно, а `+пароль: String` — ошибка дизайна?

Итого

Class Diagram — стандартный язык описания структуры системы.

Разработчики пишут по ней код, SA договаривается с командой о структуре данных до начала реализации.

Что мы умеем:

- Рисовать класс с 3 секциями и Visibility символами
- Указывать типы данных и кратность связей
- Различать 4 типа связей по символам
- Маппить Class Diagram в SQL-таблицы
- Читать чужую диаграмму и находить ошибки

Диаграмма ценна тогда, когда её читают. Рисуй так, чтобы разработчик не переспрашивал.

Что дальше

Следующие темы:

- **Activity Diagram** — диаграмма активностей (похожа на BPMN, но для одного актора)
- **Data Modeling** — ER-диаграммы, нормализация
- **ERD → SQL** — от концептуальной модели к реальной схеме
- **Sequence Diagram** — взаимодействие объектов во времени

Class Diagram и ERD — два кита документации структуры. Дальше научимся рисовать поведение системы.